

Examen JEE (Février 2026)

ISCAE – 3^{ème} année (DI, RT)
Documents non autorisés

I – GIT, Web Service Rest et Servlet

Propositions	Réponse	
Quels sont les langages WEB qu'un navigateur est capable d'interpréter/exécuter ?	PHP	
	JavaScript	
	CSS	
	JSP	
	HTML	
	Servlet	

Propositions		Réponse	
Quelle est la commande qui permet de créer un repository GIT ?	a	git create	
	b	git init	
	c	git create	
Après avoir créé un fichier, quelles sont les commandes obligatoires pour pousser ce fichier sur un repos local ?	a	git add ; git commit	
	b	git add ; git commit ; git status	
	c	git add ; git commit ; git push	
Que est la commande dont on recommande l'exécution avant chaque action GIT pour s'assurer de la situation de chaque modification ?	a	git etat	
	b	git pull	
	c	git etat	
Pour créer un Thread en Java, il faut	a	Créer une classe qui implémente l'interface Runnable	
	b	Créer une classe qui hérite de la classe Thread	
	c	Aucune de ces réponses	
En cours, nous avons étudié les applications JEE de type ?	a	Client lourd	
	b	Client léger	
	c	Aucun des deux	

1. Quels sont les différentes familles de code http. Citez-les et indiquer à quoi sert chaque famille (200 ...)

II – Git, Web Service Rest et Servlet

1^{ère} partie

Nous avons la servlet suivante (*ProductServlet.java*) dans le projet *shopApp*.

```
package com.shop.servlet;
public class ProductServlet extends HttpServlet {
```

```

private static Map<String, Product> productCatalog = new
HashMap<>();

@Override
public void init(ServletConfig config) throws
ServletException {
    String catalogName =
config.getInitParameter("catalogName");
    System.out.println("Initialisation du catalogue : " +
catalogName);

    // Initialisation avec quelques produits
    productCatalog.put("001", new Product("001", "Laptop",
1200.0));
    productCatalog.put("002", new Product("002", "Mouse",
25.0));
}

@Override
public void service(HttpServletRequest req,
HttpServletResponse resp)
throws ServletException, IOException {
    resp.setContentType("text/html");
    PrintWriter out = resp.getWriter();

    String productId = req.getParameter("id");
    String action = req.getParameter("action");

    out.println("<HTML>");
    out.println("<BODY>");

    if (action != null && action.equals("list")) {
        out.println("<h2>Liste des produits:</h2>");
        for (Product p : productCatalog.values()) {
            out.println("<p>" + p.getName() + " - " +
p.getPrice() + " EUR</p>");
        }
    } else if (productId != null &&
productCatalog.containsKey(productId)) {
        Product product = productCatalog.get(productId);
        out.println("<h2>Produit trouvé:</h2>");
        out.println("<p>ID: " + product.getId() + "</p>");
        out.println("<p>Nom: " + product.getName() +
"</p>");
        out.println("<p>Prix: " + product.getPrice() + "
EUR</p>");
    }
}

```



```

    } else {
        out.println("<p>Aucun produit trouvé</p>");
    }

    out.println("</BODY>");
    out.println("</HTML>");
    out.close();
}
}

```

Et le fichier web.xml suivant

```

<web-app>
  <display-name>shopApp</display-name>
  <servlet>
    <servlet-name>productServlet</servlet-name>
    <servlet-class>ProductServlet</servlet-class>
    <init-param>
      <param-name>catalogName</param-name>
      <param-value>Catalogue Principal</param-
value>
    </init-param>
  </servlet>
  <servlet-mapping>
    <servlet-name>productServlet</servlet-name>
    <url-pattern>/products</url-pattern>
  </servlet-mapping>
</web-app>

```

a. Afin de déclarer la servlet dans le web.xml (balise <servlet-class>), est-il obligatoire d'indiquer le nom du package ?

1. Oui. Si oui, corriger l'erreur
2. Non

b. La servlet est accessible sur quel URL (serveur local, port 8080) ?

1. <http://localhost:8080/shopApp/WEB-INF/classes/ProductServlet>
2. <http://localhost:8080/shopApp/ProductServlet>

3. <http://localhost:8080/shopApp/products>

4. <http://localhost:8080/products>

c. Que faut-il saisir comme URL dans le navigateur pour afficher la liste de tous les produits ?

1. <http://localhost:8080/shopApp/products>

2. <http://localhost:8080/shopApp/products?action=list>

3. <http://localhost:8080/shopApp/products?id=list>

4. <http://localhost:8080/shopApp/products?list=true>

d. Que faut-il saisir comme URL pour afficher les détails du produit avec l'ID "001" ?

1. <http://localhost:8080/shopApp/products?id=001>

2. <http://localhost:8080/shopApp/products?product=001>

3. <http://localhost:8080/shopApp/products/001>

4. <http://localhost:8080/shopApp/products?action=001>

e. Dans quel ordre sont exécutées les méthodes de la servlet lors du premier accès ?

1. service() puis init()

2. init() puis service()

3. init() puis destroy() puis service()

4. service() uniquement

2^{ème} partie Web service

Nous avons la structure de données suivante permettant de gérer des commandes (orders):
`Map<String, Order> orders = new HashMap<>();`

```
public class Order {  
    private String orderId;  
    private String customerName;  
    private String productId;  
    private int quantity;  
    private String status; // "pending", "confirmed", "shipped", "delivered"  
  
    public Order(String orderId, String customerName, String productId, int quantity) {  
        this.orderId = orderId;  
        this.customerName = customerName;  
    }  
}
```

```

        this.productId = productId;
        this.quantity = quantity;
        this.status = "pending";
    }
}

@Path("/orders")
public class OrderResource {

    private static Map<String, Order> orders = new HashMap<>();
}

```

Pour ajouter une commande dans cette structure, il suffit de donner son identifiant et un objet de type `Order`. Exemple :

```
orders.put("ORD001", new Order("ORD001", "Ahmed", "P001", 2));
```

La suppression se fait via la méthode « `remove` ». Par exemple `orders.remove("ORD001")` supprime la commande dont l'identifiant est "ORD001" de la structure.

L'objectif de cet exercice est de réaliser une application qui expose des services Web REST (`OrderResource`) permettant de faire les tâches suivantes. :

1. Créer une nouvelle commande
2. Supprimer une commande
3. Afficher la liste de toutes les commandes
4. Afficher une commande donnée grâce à son identifiant
5. Mettre à jour le statut d'une commande

Penser à bien apporter les modifications nécessaires à la classe `OrderResource` en respectant les bonnes pratiques du développement logiciel (utilisation appropriée des annotations JAX-RS, choix des méthodes HTTP appropriées).